Haskell – Lab 4

Prepared by Dr. Wooi Ping Cheah

Solution for the Exercises from Chapter 5 – List Comprehensions

1.

```
pyths1 :: Int -> [(Int,Int,Int)]
pyths1 n = [(x,y,z) | x <- r, y <- r, z <- r, x^2 + y^2 == z^2]
   where r = [1..n]

pyths2 :: Int -> [(Int,Int,Int)]
pyths2 n = [(x,y,z) | x <- r, y <- r, z <- r, x <= y, x^2 + y^2 == z^2]
   where r = [1..n]

pyths3 :: Int -> [(Int,Int,Int)]
pyths3 n = [(x,y,z) | x <- [1..n], y <- [x..n], z <- [y..n], x <= y, x^2 + y^2 == z^2]
```

2.

```
factors :: Int -> [Int]
factors n = [x | x <- [1..n], n `mod` x == 0]

perfects :: Int -> [Int]
perfects n = [x | x <- [1..n], x == sum [y | y <- [1..(x-1)], x `mod` y == 0]]

perfects' :: Int -> [Int]
perfects' n = [x | x <- [1..n], x == sum (factors x) - x]
```

3.

```
scalar :: [Int] -> [Int] -> Int
scalar xs ys = sum [x*y | (x,y) <- zip xs ys]

scalar' :: Num a => [a] -> [a] -> a
scalar' xs ys = sum [x*y | (x,y) <- zip xs ys]
```

Solution for the Exercises from Chapter 6 – Recursive Functions

1.

```
and :: [Bool] -> Bool
and [] = True
and (x:xs) = x && and xs

concat :: [[a]] -> [a]
concat [] = []
concat (xs:xss) = xs ++ concat xss

replicate :: Int -> a -> [a]
replicate 0 x = []
replicate n x = x : (replicate (n-1) x)

(!!) :: [a] -> Int -> a
(x:xs) !! 0 = x
(x:xs) !! n = xs !! (n-1)
```

```
elem :: Eq a => a -> [a] -> Bool
elem n xs | null xs      = False
          | n == head xs = True
          | otherwise    = elem n (tail xs)

elem' :: Eq a => a -> [a] -> Bool
elem' x [] = False
elem' x (y:xs) | x == y = True
               | otherwise = elem x xs
```

2.

```
merge :: Ord a => [a] -> [a] -> [a]
merge []     ys     = ys
merge xs     []     = xs
merge (x:xs) (y:ys) | x < y     = x : (merge xs (y:ys))
                    | otherwise = y : (merge (x:xs) ys)
```

3.

```
msort :: Ord a => [a] -> [a]
msort [] = []
msort [x] = [x]
msort xs = merge (msort (take n xs)) (msort (drop n xs))
           where n = (length xs) `div` 2
```

Exercises
from
Chapter 7 – Higher-Order Functions

Write a function ***squarePositives*** using ***map*** and ***filter*** that takes a list of integers and returns a new list where all negative numbers are removed and the remaining (non-negative) numbers are squared.

Example:

> squarePositives [-2, 3, -1, 4, 0, 5]

[9, 16, 0, 25]

```
squarePositives :: [Int] -> [Int]
squarePositives xs = [redacted] (^2) ([redacted] (>= 0) xs)
```

Example:

```
> squarePositives [-2, 3, -1, 4, 0, 5]
[9, 16, 0, 25]
```

```
squarePositives2 :: [Int] -> [Int]
squarePositives2 = [redacted] (^2) . [redacted] (>= 0)
```

Example:

```
> squarePositives2 [-2, 3, -1, 4, 0, 5]
[9, 16, 0, 25]
```

Write a function ***capitalizeLong*** that takes a list of strings, and a length ***n*** (integer). Only words longer than ***n*** chars are uppercased.

Example:

> capitalizeLong 3 ["hi", "hello", "world", "hs"]

["HELLO", "WORLD"]

(Only words longer than 3 chars are uppercased)

```
import Data.Char (toUpper)
capitalizeLong :: Int -> [String] -> [String]
capitalizeLong n = [redacted] (map toUpper) . [redacted] (\x -> length x > n)

-- Note: map toUpper works on a single string (e.g., "hello" -> "HELLO")
-- Note: "hello" is equivalent to [‘h’, ‘e’, ‘l’, ‘l’, ‘o’]
```

Example:

```
> capitalizeLong 3 ["hi", "hello", "world", "hs"]
["HELLO", "WORLD"]
```

(1) What are higher-order functions that return functions as results better known as?

Hints:

You can find the answer to this question from the following videoclip:

Haskell for Imperative Programmers #7 - Partial Function Application & Currying

https://www.youtube.com/watch?v=m12c99qgHBU

(2) Express the comprehension [f x | x ← xs, p x] using the functions map and filter.

Hints:

This is a simple one-line code

```
ghci>
ghci> func1 f p xs = [f x | x <- xs, p x]
ghci> func1 (+1) even [1,2,3,4,5]
[3,5]
ghci> func2 f p xs = [redacted] f ([redacted] p xs)
ghci> func2 (+1) even [1,2,3,4,5]
[3,5]
ghci>
```

Fill in the blanks

func1 and func2 are two higher-order functions we defined – one using list comprehension, the other using map and filter functions.

(3) Redefine map f and filter p using foldr.

Define map f using foldr

map :: (a -> b) -> [a] -> [b]

map f ys = foldr ( ) [] ys

This is a lambda function that accumulates f x, where x is the head of the list ys

Define filter p using foldr

filter :: (a -> Bool) -> [a] -> [a]

filter p ys = foldr ( ) [] ys

This is a lambda function that accumulates x if p x is true, where x is the head of the list ys

3.

```
ghci>
ghci> map' f ys = foldr (                    ) [] ys
ghci> map' (+1) [1,2,3,4,5]
[2,3,4,5,6]
ghci> filter' p ys = foldr (                                        ) [] ys
ghci> filter' even [1,2,3,4,5]
[2,4]
ghci>
```

Fill in the blanks

Exercises
from
Chapter 8 – Declaring Types and Classes

Define a data type ***Employee*** that can represent:

- A ***FullTime*** employee with a name (***String***) and annual salary (***Float***).
- A ***Contractor*** with a name (***String***) and hourly rate (***Float***).

Then, write a function ***monthlyPay :: Employee -> Float*** that calculates the monthly pay:

For ***FullTime***, divide the annual salary by 12.

For ***Contractor***, assume 160 work hours per month (hourly rate × 160).

```
data Employee = FullTime String Float | Contractor String Float

monthlyPay :: Employee -> Float
monthlyPay (FullTime _ salary) = [redacted] / 12
monthlyPay (Contractor _ rate) = [redacted] * 160


> monthlyPay (FullTime "Alice" 60000.0)
5000.0

> monthlyPay (Contractor "Bob" 25.0)
 4000.0
```

(1) Using recursion and the function add, define a function that multiplies two natural numbers.

Hints:

You are given the type declaration for Nat, and function definition for add':

```
data Nat = Zero | Succ Nat
        deriving Show

add' Zero n = n
add' (Succ m) n = Succ (add' m n)
```

Write a function that multiplies two natural numbers.

You can call this function as mult', which will use the add' function, and is quite similar in pattern to add' function.

```
ghci> mult' (Succ(Succ Zero)) (Succ(Succ(Succ Zero)))
Succ (Succ (Succ (Succ (Succ (Succ Zero)))))
```

References
Functional Programming & Haskell

foldr explained

Definition for foldr with a simple example

https://www.youtube.com/watch?v=Dmd6Q2i7gdw

Foldr

Application of foldr with simple examples

https://www.youtube.com/watch?v=cyGtltWKWQg